

PYTHON PROJECT FOR CHESS APPLICATION

Chess Application with Flask and MySQL

Project Overview

This project is a Chess Web Application developed using Flask as the backend framework and MySQL as the database. It allows users to play chess against each other, track game history, and manage user profiles. The application features a real-time chess board, user authentication, and the ability to save, load, and review past games.

Key Features:

- User authentication (Login/Register)
- Real-time Chess game interface
- Save and load game functionality
- Game history (view past games)
- Database-backed user profiles and game tracking

Technologies Used:

- Flask: Web framework for Python
- MySQL: Database management system to store user data, game history, and other necessary information
- HTML/CSS/JavaScript: Frontend for rendering the chess board and game interface
- Flask-SQLAlchemy: ORM for interacting with MySQL database
- PyChess: Chess logic library for handling game rules and moves (or custom chess logic)
- Socket.IO: Real-time updates for two-player gameplay
- Bootstrap: Frontend UI framework

Installation

Prerequisites:

Ensure you have the following installed:

- Python 3.8 or higher
- MySQL server
- pip (Python package manager)

Step 1: Clone the Repository

bash

```
git clone https://github.com/your-repo/chess-flask-mysql.git
```

```
cd chess-flask-mysql
```

Step 2: Install Dependencies

Create a virtual environment and install required Python libraries.

bash

```
python3 -m venv venv
```

```
source venv/bin/activate # On Windows, use venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Step 3: Set Up MySQL Database

1. Create a MySQL database:

sql

```
CREATE DATABASE chess_app;
```

2. Create tables for users, games, and game history:

sql

```
CREATE TABLE users (  
  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  username VARCHAR(100) NOT NULL UNIQUE,  
  
  email VARCHAR(100) NOT NULL,  
  
  password VARCHAR(255) NOT NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
  
);
```

```
CREATE TABLE games (  
  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  white_player_id INT,  
  
  black_player_id INT,  
  
  winner INT,  
  
  moves TEXT,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (white_player_id) REFERENCES users(id),  
  
  FOREIGN KEY (black_player_id) REFERENCES users(id) );
```

Step 4: Configure the Application

Edit the configuration settings for your Flask app (usually found in `config.py` or `app.py`).

- Set up MySQL connection details:

python

SQLALCHEMY_DATABASE_URI = 'mysql://username:password@localhost/chess_app'

SQLALCHEMY_TRACK_MODIFICATIONS = False

Step 5: Run the Flask Application

Start the development server with:

bash

python app.py

The application will be available at `http://localhost:5000`.

Project Structure

The following is an overview of the project's directory structure:

/chess-flask-mysql

```
|— app.py          # Main Flask application file
|— config.py       # Configuration file for Flask and database
|— models.py       # Database models (User, Game)
|— /static         # Static files (CSS, JS, images)
|   |— /css
|   |— /js
|   |— /images
|— /templates      # HTML templates
|   |— index.html
|   |— game.html
|   |— login.html
|   |— register.html
|— requirements.txt # Python dependencies
```

Key Files:

- app.py : The main file where the Flask application is initialized and routes are defined.
- config.py: Contains configuration settings such as database URI, secret keys, etc.
- models.py: Contains the SQLAlchemy ORM models for users and games.
- static: Directory for CSS, JS, and image files used in the frontend.
- templates: Directory for HTML files, which are rendered in the browser.

Flask Routes

1. Home Route (`/`)

The main page that displays the chessboard and links to user authentication routes.

python

```
@app.route('/')
```

```
def home():
```

```
    return render_template('index.html')
```

2. Register Route (`/register`)

Allows new users to register with a username, email, and password.

python

```
@app.route('/register', methods=['GET', 'POST'])
```

```
def register():
```

```
    if request.method == 'POST':
```

```
        username = request.form['username']
```

```
        email = request.form['email']
```

```
        password = request.form['password'] # Use hashing for security
```

```
        user = User(username=ROOT, email=email, password=ROOT)
```

```
        db.session.add(user)
```

```
        db.session.commit()
```

```
flash('Account created successfully!')

return redirect(url_for('login'))

return render_template('register.html')
```

3. Login Route (`/login`)

Allows users to log in and access their profile or start a new game.

python

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = User.query.filter_by(username=ROOT).first()
        if user and user.check_password(ROOT):
            session['user_id'] = user.id
            flash('Logged in successfully!')
            return redirect(url_for('home'))
        flash('Invalid credentials, please try again.')
    return render_template('login.html')
```

4. Start Game Route (`/game/<game\_id>`)

Displays the chess game board and allows two users to play.

python

```
@app.route('/game/<game_id>')  
  
def game(game_id):  
    game = Game.query.get(game_id)  
    return render_template('game.html', game=game)
```

MySQL Models

User Model (`models.py`)

python

```
class User(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    username = db.Column(db.String(100), unique=True, nullable=False)  
    email = db.Column(db.String(100), unique=True, nullable=False)  
    password = db.Column(db.String(255), nullable=False)  
    created_at = db.Column(db.DateTime, default=datetime.utcnow)  
  
    def check_password(self, password):  
        return self.password == password # Add hashing here for production
```


Game Model (`models.py`)

python

class Game(db.Model):

id = db.Column(db.Integer, primary_key=True)

white_player_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)

black_player_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)

winner = db.Column(db.Integer) # ID of the winner player

moves = db.Column(db.Text, nullable=False) # List of moves in PGN format

created_at = db.Column(db.DateTime, default=datetime.utcnow)

Running the Application

1. Navigate to the project directory.
2. Install the dependencies.
3. Set up MySQL and configure the database.
4. Start the Flask server:

bash

python app.py

5. Open the browser and visit `http://localhost:5000` to access the application.

Conclusion

This chess web application built with Flask and MySQL provides a full-fledged user interface for playing chess, registering, logging in, and tracking game history. You can extend this app by adding more features like real-time multiplayer using Socket.IO, more advanced chess logic with a library like python-chess, or by implementing advanced features like online matchmaking.